**《大模型背景下面向对象程序设计课程教学探索》，作者：钱 诚 徐则中 游庆祥，发表于[科教文汇,2026] 使用 kimi 大模型**

**《大模型背景下面向对象程序设计课程教学探索》导读**

**一、研究背景与动机：当 ChatGPT 遇上编程课**

想象一下这个场景：一位计算机专业的大二学生正在完成 Java 作业，面对"设计一个学生成绩管理系统"的题目，他打开 ChatGPT，输入提示词，几秒钟后，一段完整的代码就生成了。复制、粘贴、提交——作业完成了，但他真的学会了面向对象设计吗？

这正是当前高校程序设计课程面临的**真实困境**。随着 ChatGPT、DeepSeek 等大语言模型的普及，传统编程教学模式的根基正在被动摇：

**痛点一：学生的"能力幻觉"** 过去，学生需要通过反复调试、查阅文档、手写代码来掌握编程技能。现在，大模型能直接生成可运行的代码，导致许多学生跳过思考过程，对 AI 产生过度依赖。表面上看作业完成了，实际上连基础的类设计、继承多态等核心概念都没理解。

**痛点二：一刀切的教学困境** 传统课堂中，教师通常用同样的案例和作业要求对待所有学生。但学生的编程基础差异巨大：有的已经能独立开发小项目，有的连基础语法都磕磕绊绊。这种"大锅饭"式教学让优等生觉得无聊，后进生又跟不上。

**痛点三：课程目标的错位** 传统课程目标侧重于"能写出正确的代码"，但在 AI 时代，**写代码本身正在贬值**。企业更需要的是软件架构设计能力、需求分析能力，以及判断 AI 生成代码质量的能力。现有课程体系未能及时回应这种转变。

这篇论文的作者——常州工学院的钱诚、徐则中、游庆祥三位老师——敏锐地捕捉到了这些变化。他们没有选择"鸵鸟策略"（禁止学生用 AI），也没有放任自流，而是提出了一种**"融入大模型"**的全新教学思路：把 AI 从"作弊工具"重新定位为"学习助手"，并围绕这一定位重构整个课程体系。

---

**二、核心方法：三层重构框架**

论文的核心贡献可以概括为**"三个重构"**，形成一个完整的教学改革框架：

**1. 课程目标重构：从"写代码"到"建模型"**

作者对比了传统目标与重构后的目标（见表 1）：

**表 1 原有课程目标与重构课程目标对比**

| 课程目标 | 原有课程目标 | 重构课程目标 | 差异性 |
|---|---|---|---|
| 目标1 | 掌握面向对象的基本概念，具备使用面向对象技术进行问题抽象和表达的能力 | 理解面向对象核心概念，掌握面向对象分析与设计方法，能够运用面向对象方法进行问题域建模与抽象表达 | 强调对设计原则的理解和建模应用 |
| 目标2 | 掌握面向对象程序设计领域的文件处理、多线程编程、数据库访问等应用开发技术 | 能够在大模型的辅助下将文件I/O、多线程、数据库访问等应用问题分解为合理的对象模型 | 从掌握应用技术转向应用技术的分解、软件系统建模 |
| 目标3 | 掌握面向对象程序设计开发工具，具备使用面向对象开发工具对具体问题进行开发设计的能力 | 具备在大模型工具辅助下进行面向对象程序的开发、调试和重构的能力，能够分析、判断大模型生成的代码是否符合面向对象设计原则 | 强调大模型在编码过程中的辅助工具角色和合理运用 |

（图注：该表从 3 个核心维度对比了面向对象程序设计课程重构前后的目标差异：

- 目标 1：从基础概念掌握升级为强调设计原则理解与领域建模能力；
- 目标 2：从应用技术掌握转向技术问题分解与软件系统建模；
- 目标 3：新增大模型辅助开发的要求，强调对 AI 生成代码的分析与设计原则校验。

通过对比清晰呈现了课程从“技术应用”向“建模思维 + AI 协作能力”转型的核心方向。）关键转变在于：课程不再要求学生"从零写每一行代码"，而是强调**软件建模能力**和 **AI 协作能力**。学生需要学会的是：如何向 AI 描述需求、如何评估 AI 生成的代码、如何将 AI 片段整合到合理的架构中。

**2. 课程内容重构："概念—设计模式—工程实践"三级案例库**

这是最具操作性的创新。作者设计了一个**分层递进的案例体系**：

- **概念阶段**：聚焦基础（类、对象、封装、继承、多态），案例简单，代码量少
- **设计模式阶段**：引入单例、工厂、观察者等常用模式，要求学生在给定模式下完成设计
- **工程实践阶段**：贴近真实项目，学生需自主划分模块、选择设计模式、用 AI 辅助实现具体代码

**差异化教学机制**：通过课前测试将学生分为基础级、进阶级、高阶级。基础级学生重点完成概念阶段任务，进阶级侧重设计模式应用，高阶级则挑战完整工程案例。每个级别都可以**按需调用大模型**，但使用方式不同——基础级用 AI 辅助理解语法，高阶级用 AI 加速编码但自主把控架构。

**表2　学生能力分层与三阶段案例设置关系**

| 学生能力级别 | 案例阶段 | | |
|---|---|---|---|
| | 概念阶段 | 设计模式阶段 | 工程实践阶段 |
| 基础级 | 类和对象、封装、继承和多态等概念 | — | — |
| 进阶级 | — | 单例模式、工厂模式、观察者模式、适配器模式、装饰器模式等设计模式的运用 | — |
| 高阶级 | — | — | 贴近实际的工程实践案例 |

（图注：该表展示了面向对象程序设计课程中，学生能力分层与三阶段案例的匹配关系：

- **基础级**：聚焦概念阶段，学习类和对象、封装、继承、多态等核心面向对象基础概念；
- **进阶级**：聚焦设计模式阶段，实践单例、工厂、观察者等经典设计模式的运用；
- **高阶级**：聚焦工程实践阶段，完成贴近实际场景的工程化案例开发。

通过分层案例设计，实现从基础概念到设计模式再到工程实践的梯度化能力培养。）

表3　课内实验任务设置

| 实验名称 | 实验内容 | 实验子任务 |
| --- | --- | --- |
| 开发环境配置 | JDK安装与环境变量配置 | 下载并安装JDK、配置环境变量以及验证安装(概念阶段) |
| | | 安装、配置IDE(设计模式阶段) |
| | | 编写、运行测试程序,验证开发环境(工程实践阶段) |
| 类和对象 | 学生成绩管理系统的设计和实现 | 学生类、课程类的属性封装和程序实现(概念阶段) |
| | | 工厂设计模式下学生类、课程类的设计和实现(设计模式阶段) |
| | | 学生成绩和课程模块划分、设计与实现(工程实践阶段) |
| 输入输出 | 学生成绩管理系统的设计和实现 | 学生成绩输入流和输出流的程序实现(概念阶段) |
| | | 策略设计模式下学生信息、课程信息序列化和反序列化的设计和实现(设计模式阶段) |
| | | 学生成绩、课程信息读写的可扩展接口设计与实现(工程实践阶段) |
| 继承和多态 | 饮品点单系统的设计和实现 | 咖啡类、茶类等饮品基类和派生类的程序实现(概念阶段) |
| | | 装饰器设计模式下的饮品类层次结构的设计和实现(设计模式阶段) |
| | | 饮品点单和结算系统模块划分、设计与实现(工程实践阶段) |
| 多线程编程 | 简易服务器的设计和实现 | http请求响应程序的实现(概念阶段) |
| | | 生产者—消费者模式下的http请求响应程序的设计与实现(设计模式阶段) |
| | | 简易服务器通信层、协议层、业务逻辑层的划分、定义和实现(工程实践阶段) |

（图注：该表展示了面向对象程序设计课程的课内实验任务体系，按**开发环境配置、类和对象、输入输出、继承和多态、多线程编程**五大实验主题，拆解为**概念阶段、设计模式阶段、工程实践阶段**三个梯度子任务：

- 概念阶段：聚焦基础语法与核心概念实现（如类封装、I/O 流基础、多线程基础）；
- 设计模式阶段：引入单例、工厂、装饰器等经典设计模式，强化面向对象设计思维；
- 工程实践阶段：侧重模块划分、系统架构与可扩展接口设计，贴近真实工程场景。

通过阶梯式任务设计，实现从基础技能到设计思维再到工程能力的递进培养。）

**3. 课程考核重构：差异化评分系数**

为了激励学生挑战更高难度，作者设计了**弹性评分机制**：

- 概念阶段系数：0.75
- 设计模式阶段系数：0.85
- 工程实践阶段系数：1.0

这意味着：完成同样质量的作业，选择工程实践阶段的学生能获得更高分数。这种设计**用成绩杠杆引导学生跳出舒适区**，主动挑战更复杂的任务。

---

## 三、主要结果：教学改革成效如何？

作者在常州工学院进行了**对照实验**：

- **对照组**：2022 级一班（40 人），采用传统教学模式，未引入大模型
- **实验组**：2023 级一班（38 人），采用重构后的教学模式，允许使用 DeepSeek、通义千问等工具

实验结果（见图 1）呈现出有趣的**"两升一降"**：

**提升方面**：

- **课程目标 1**（设计原则理解）：实验组达成度 0.80 vs 对照组 0.73，**提升 9.6%**
- **课程目标 3**（AI 协作与代码评估）：实验组达成度 0.70 vs 对照组 0.62，**提升 12.9%**

这表明新教学模式确实增强了学生的软件设计能力和 AI 工具使用能力。

**下降方面**：

- **课程目标 2**（类库熟悉程度）：实验组达成度 0.77 vs 对照组 0.85，**下降 9.4%**

作者分析原因：由于大模型能自动生成类库调用代码，学生手动查阅 API 文档的机会减少，对底层类库的熟悉程度反而下降。这提示我们：**AI 辅助是把双刃剑，需要刻意保留某些"手动"训练环节**。

总体而言，实验验证了"融入大模型"策略的有效性，但也揭示了需要警惕的新问题。

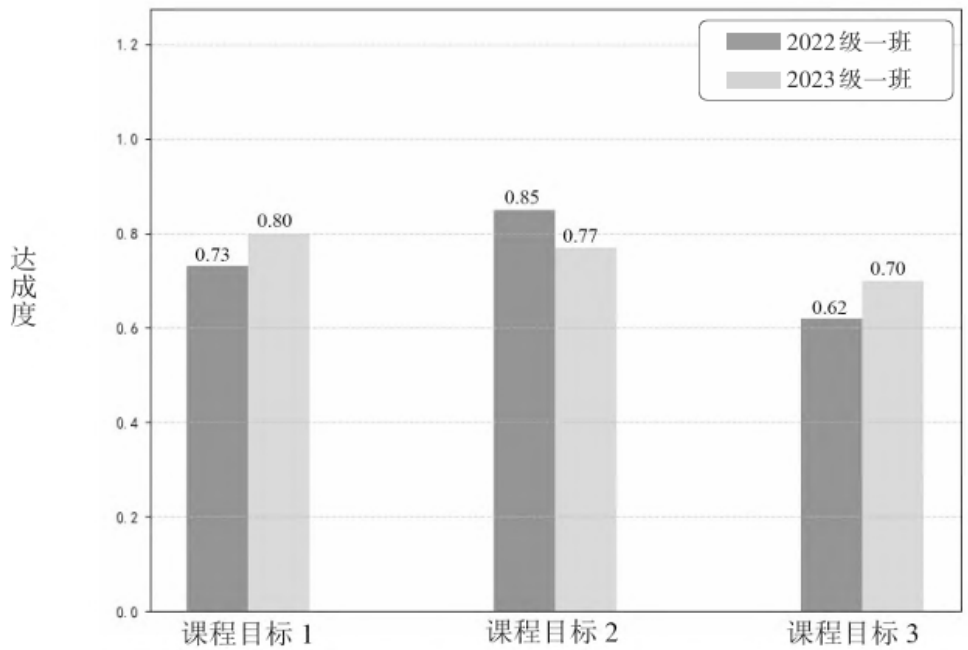


图1　某高校计算机科学与技术专业2022级一班与2023级一班学生课程目标达成度对比

（图注：该柱状图对比了两个班级在三项课程目标上的达成度：

- 课程目标 1：2022 级一班达成度为 0.73，2023 级一班为 0.80，后者更高；
- 课程目标 2：2022 级一班达成度为 0.85，2023 级一班为 0.77，前者更高；
- 课程目标 3：2022 级一班达成度为 0.62，2023 级一班为 0.70，后者更高。

整体来看，2023 级一班在目标 1 和目标 3 上表现更优，体现出课程改革后在设计建模与大模型辅助开发能力上的提升；2022 级一班在目标 2 上略占优势，反映出传统应用技术掌握的基础优势。）

**四、个人小结：启示与思考**

**优点**

**1. 务实的态度** 面对 AI 浪潮，很多教师陷入"恐慌"（担心学生作弊）或"盲目乐观"（认为 AI 能解决一切）。本文作者展现了难得的务实：既不否认大模型的工具价值，也不放弃教师的设计引导责任，而是**重新定义了人机协作的边界**。

**2. 可操作性强** 论文提供了完整的实施路径：从目标设定、案例分级、学生分层到考核系数，每一步都有具体做法。其他高校教师可以直接借鉴，不需要额外的技术投入。

**3. 问题导向的实验设计** 对照实验不仅展示了成果，也诚实暴露了问题（类库熟悉度下降）。这种"报喜也报忧"的态度增强了研究的可信度，也为后续改进指明了方向。

**局限与启发**

**1. 样本规模较小** 两个班级共 78 人，且来自同一所学校，结论的普适性有待更大规模验证。不同层次高校（985/211 vs 应用型本科）的学生基础差异很大，分层标准可能需要调整。

**2. 长期效果未知** 实验只覆盖一个学期，学生在使用 AI 辅助后，**长期编程能力**（如毕业设计、工作中的独立开发）是否会受到影响？这需要跟踪研究。

**3. 对"创造力"的关注不足** 当前框架侧重"设计能力"和"工具使用能力"，但对**创新性编程**（如

算法优化、架构创新）的培养路径不够清晰。当 AI 能生成常规代码后，人类程序员的核心竞争力究竟在哪里？这个问题值得更深入探讨。

**对初学者的建议**

如果你是一名正在学习编程的学生，这篇论文给你三点启示：

1. **不要迷信 AI 的"全知全能"**：大模型擅长生成代码片段，但软件的整体架构、需求分析、可维护性设计仍需人类把控。学会"向 AI 提问"和"评判 AI 输出"比"让 AI 代写"更重要。
2. **主动选择"困难模式"**：论文的评分系数设计告诉我们，成长往往发生在舒适区边缘。遇到编程任务时，先尝试自主设计，再让 AI 辅助实现，这样收获最大。
3. **重视基础概念**：实验中学生对类库熟悉度下降的结果提醒我们，AI 生成的代码可能让我们"知其然而不知其所以然"。建议定期脱离 AI 工具，手写一些基础代码，保持对技术细节的敏感度。

---

**结语**

这篇论文的价值不仅在于提出了一套教学改革方案，更在于它回应了一个时代命题：**当 AI 开始替代人类的某些技能时，教育应该教什么、怎么教？** 作者给出的答案是：从"教知识"转向"教能力"，从"防 AI"转向"用 AI"，从"统一教学"转向"因材施教"。这些思路不仅适用于编程教学，也为其他面临 AI 冲击的学科提供了借鉴。